Deep Web Research — 2026-07-02

Standing operator directive: "*Perform frequent deep web researches about solutions for all of our challenges, new ideas, game-changing approaches, and various opensource codebase and services we can incorporate into our System.*"

Every claim below traces to a real URL that was fetched or searched during this pass (see the per-topic **Sources** lists). No source-code was edited, no Gradle was run, no secrets were read while producing this report.

Topic 1 — Brotli decompression on Android (Ktor / OkHttp)

Context (our bug)

We root-caused a total-provider-breakage: setting a **manual** `Accept-Encoding: gzip, deflate, br` header on a Ktor **OkHttp-engine** client made OkHttp return **raw compressed bytes**. OkHttp only *transparently* decompresses an encoding it negotiated **itself**; the moment the application sets `Accept-Encoding`, OkHttp assumes the caller will handle decompression and stops stripping `Content-Encoding`. rutracker.org replies `Content-Encoding: br`, so we got brotli bytes fed into a windows-1251 HTML parser = garbage. Current landed fix (commit `8bb2317c`): drop the manual header so OkHttp negotiates + decompresses gzip itself.

How OkHttp actually behaves

- OkHttp's `BridgeInterceptor` adds `Accept-Encoding: gzip` and transparently decompresses the response **only when the app did not set `Accept-Encoding`**.
- `okhttp-brotli`'s `BrotliInterceptor` extends that transparent behaviour to `br`: it "adds `Accept-Encoding: br` to request and checks (and strips) for `Content-Encoding: br` in responses … provided `Accept-Encoding` is not set previously." It is response-only ("not used for sending requests"). Source: okhttp-brotli README + `BrotliInterceptor` API docs.

=> The same "don't set the header yourself" rule that fixed our gzip path is exactly what makes `BrotliInterceptor` work. Our current fix is correct; adding brotli is an **enhancement**, not a required fix.

Option A — `okhttp-brotli` `BrotliInterceptor` (recommended shape, with a caveat)

```
// gradle/libs.versions.toml
//   okhttp-brotli = { module = "com.squareup.okhttp3:okhttp-
        brotli", version.ref = "okhttp" }
// Latest: 5.4.0 (README, master). Keep version.ref aligned with
        the okhttp BOM/core
// version already in the catalog so brotli and core never
        diverge.

// Ktor OkHttp engine config:
HttpClient(OkHttp) {
    engine {
        addInterceptor(okhttp3.brotli.BrotliInterceptor.INSTANCE)
    }
}
```

- Do **NOT** also set a manual `Accept-Encoding` anywhere (repeats our bug).
- Ships a bundled pure-JVM brotli **decoder** (`org.brotli:dec`) transitively, so no NDK/native `.so` per-ABI — friendly to a slim APK.

⚠ Security finding — CVE-2023-3782 (brotli decompression bomb) — HAS NO FIX

- `okhttp-brotli`'s `intercept()` does **not** guard against decompression bombs: "a file weighing several KBs can be decompressed into 10GB", crashing the process via OOM. Triggerable by a malicious server **or a MitM injecting a brotli zip-bomb** into any HTTP response.
- Snyk (checked the `okhttp-brotli 4.12.0` package page directly) states: **"There is no fixed version for `com.squareup.okhttp3:okhttp-brotli`"**, affected range `[0,)` — i.e. **every version including 5.x is still vulnerable.** NVD's CPE range is also open-ended (`version 0, lessThan *`).
- Implication for Lava: rutracker/rutor/nnmclub/kinozal are third-party origins reached over `usesCleartextTraffic="true"` + they've been behind Cloudflare — a hostile middlebox injecting a br bomb is a realistic threat model. If we adopt `BrotliInterceptor` we MUST cap the decompressed size ourselves (wrap the interceptor / enforce a max-body limit on the response stream) and record a §6.O/§6.T.4 note. Given the

app already works fine on plain gzip, **brotli is not worth this risk unless a provider becomes br-only.**

Option B — Ktor `ContentEncoding` plugin

- Ktor's `ContentEncoding` client plugin supports **gzip, deflate, identity only — brotli is NOT supported.** Kotlin Slack + Ktor issue #408 confirm no built-in br encoder/decoder; the documented workaround is to implement the `ContentEncoder` interface yourself and register it via `customEncoder()`. Not worth it vs. the engine-level interceptor.

Option C — lower-level brotli libs (only if we hand-roll)

- `com.aayushatharva.brotli4j:brotli4j:1.23.0` — native JNI (C brotli via bundled per-OS/arch `.so`, JAR ~2MB per platform). Fast, but adds native libs / R8-keep surface and APK bloat — **overkill for Android**.
- `org.brotli:dec` (Google, pure-Java decoder) — smallest footprint, `InputStream` API only (blocking). This is what `okhttp-brotli` already pulls in transitively.
- (Reference: Methanol HTTP client documents the same brotli landscape.)

Recommended for Lava (Topic 1)

1. **Keep the current fix** (no manual `Accept-Encoding`; OkHttp negotiates gzip). It is correct and sufficient today — verified against the OkHttp transparent-encoding contract.
2. Add a small **contract/regression test** (§6.A / Fifth Law) that asserts: with a MockWebServer replying `Content-Encoding: br` **and** no app-set `Accept-Encoding`, the client either (a) correctly decodes when `BrotliInterceptor` is installed, or (b) never silently returns raw br bytes. This locks the regression that `8bb2317c` fixed.
3. **Only if** a provider goes br-only: install `okhttp3.brotli.BrotliInterceptor` at the engine level **plus a max-decompressed-size guard** (CVE-2023-3782 has no upstream fix), and file a §6.O forensic note. Prefer the transitive `org.brotli:dec` decoder; avoid `brotli4j` native libs.
4. R8/ProGuard: `okhttp-brotli` + `org.brotli:dec` are plain JVM; no reflection-keep rules needed beyond OkHttp's existing consumer rules. `brotli4j` WOULD need keep rules for JNI + native-lib extraction — another reason to avoid it.

Sources (Topic 1)

- https://github.com/square/okhttp/blob/master/okhttp-brotli/README.md
- https://square.github.io/okhttp/5.x/okhttp-brotli/okhttp3.brotli/-brotli-interceptor/index.html
- https://github.com/square/okhttp/issues/3706
- https://ktor.io/docs/client-content-encoding.html
- https://github.com/ktorio/ktor/issues/408
- https://security.snyk.io/vuln/SNYK-JAVA-COMSQUAREUPOKHTTP3-5789026
- https://security.snyk.io/package/maven/com.squareup.okhttp3%3Aokhttp-brotli/4.12.0
- https://research.jfrog.com/vulnerabilities/okhttp-client-brotli-dos/
- https://nvd.nist.gov/vuln/detail/CVE-2023-3782
- https://github.com/hyperxpro/Brotli4j
- https://mvnrepository.com/artifact/com.aayushatharva.brotli4j/brotli4j
- https://mizosoft.github.io/methanol/brotli/

Topic 2 — Russian tracker anti-bot & session handling (2025-2026)

Cloudflare on rutracker.org

- rutracker.org has been behind Cloudflare (confirmed by the qBittorrent-RuTracker plugin issue #41: "rutracker implemented Cloudflare protection"; broke org/.nl/.net mirrors for credential logins). This is the same class of failure our own Cloudflare-mitigation work (commit `f7d0a62`, C03) targeted.
- The `cf_clearance` cookie is **bound to the exact User-Agent + IP** that solved the challenge. Reusing it from a different UA or egress IP → immediate block. So our session store must pin UA+cookie together and never rotate UA mid-session.
- Practical bypass building blocks used by the scraping ecosystem (open source):
  - **FlareSolverr** — proxy that spins a real (undetected) browser, solves the JS challenge, returns `cf_clearance` + UA to reuse. Heavy (full browser) but the de-facto standard; Prowlarr/Jackett integrate with it.
  - **Byparr** — newer, actively-maintained FlareSolverr-compatible drop-in.
  - **cloudscraper** (Python) — lightweight, handles the older JS-challenge form; less effective against current Turnstile/managed-challenge.
- Session-reuse pattern: solve once, persist `cf_clearance` + session cookies + UA, replay for subsequent requests until the

cookie expires (short-lived, often ~30 min to a few hours), then re-solve. This maps onto Lava's existing auth/session storage.

Charset gotcha (windows-1251)

- rutracker/rutor/kinozal/nnmclub serve **windows-1251** (Cyrillic), not UTF-8. Bytes must be decoded as cp1251 **before** HTML parse — decoding as UTF-8 mangles Cyrillic titles. This compounds with Topic 1: br/gzip must be decompressed FIRST, then the decompressed bytes decoded as windows-1251. (Our bug fed still-compressed bytes into the cp1251 decoder → double garbage.) Worth an explicit test asserting a known Cyrillic title round-trips through decompress→cp1251→parse.

Open-source clients/definitions to learn from

- **Prowlarr** `Prowlarr/Indexers` — Cardigann **YAML** definitions for rutracker, rutor, kinozal, nnmclub (NoNameClub). These are the canonical, community-maintained descriptions of each tracker's login flow, search rows, category maps, and (critically) the exact **headers + encoding** each site needs. Best single reference for field-level parsing parity. `Prowlarr/Indexers` issue #370 documents Jackett↔Prowlarr YAML differences; issues #1391 (kinozal invalid-torrent) and #1816 (nnmclub connect) are live troubleshooting threads.
- **Jackett** — the older C# indexer set (some trackers still only have C# impls; see Prowlarr issue #29 "parity with Jackett C# indexers").
- **`msergein/ru-cardigann-yml`** — a community repo of Russian-tracker Cardigann YAMLs (handy diff target for header/charset specifics).
- Sonarr forum thread "Cannot configure Rutor and Rutracker indexers" documents the real-world header/login pitfalls users hit.

Recommended for Lava (Topic 2)

1. **Mine the Prowlarr Cardigann YAMLs** for rutracker/rutor/kinozal/nnmclub as the source-of-truth for required request headers, login form fields, category maps, and charset. Port field-by-field into our SDK plugins; add a parity/fixture test per tracker (§6.D behavioral coverage).
2. **Pin UA↔cookie together** in the session store; never rotate UA while a `cf_clearance` is live. Treat `cf_clearance` as short-lived; re-solve on 403.
3. Add an explicit **decompress-then-cp1251** ordering test with a known Cyrillic fixture per provider — directly guards the class of bug we just shipped.
4. Consider a **FlareSolverr/Byparr sidecar** option (containerized via the Containers submodule) for the rutracker

Cloudflare path in `lava-api-go`, gated behind config — keeps the heavy browser out of the Android client. This is the same shape as our existing proxy-scrapes-upstream architecture.
5. Keep live-capture fixtures fresh (§6.D) — tracker HTML + Cloudflare posture drift.

Sources (Topic 2)

- https://github.com/nbusseneau/qBittorrent-RuTracker-plugin/issues/41
- https://www.zenrows.com/blog/cf-clearance
- https://scrapfly.io/blog/posts/how-to-bypass-cloudflare-with-flaresolverr
- https://scrapeops.io/web-scraping-playbook/how-to-bypass-cloudflare/
- https://pypi.org/project/cloudscraper/
- https://github.com/Prowlarr/Indexers/issues/370
- https://wiki.servarr.com/en/prowlarr/cardigann-yml-definition
- https://github.com/msergein/ru-cardigann-yml
- https://github.com/Prowlarr/Prowlarr/issues/29
- https://github.com/Prowlarr/Prowlarr/issues/1391
- https://github.com/Prowlarr/Prowlarr/issues/1816
- https://forums.sonarr.tv/t/cannot-configure-rutor-and-rutracker-indexers/27037

Topic 3 — Emulator-in-container + video recording + LLM-vision autonomous-QA loop

3a. Emulator-in-container (fits our Containers submodule + §6.X/§6.AH)

Two families, and which one solves our KVM problem:

- **Redroid (Remote-Android)** — containerized Android that runs the Android userspace **directly on the host Linux kernel via binder/ashmem; NEVER touches `/dev/kvm`.** Multi-arch (arm64 + amd64), GPU-optional, ships images **Android 8.1 → Android 16** (`redroid/redroid` on Docker Hub, `..._64only` arm64 variants). This is the **game-changer for our standing §6.X-debt / §6.AH-debt**: our blocker has always been "podman VM has no `/dev/kvm`/HVF". Redroid sidesteps KVM entirely — but note it is **not** the Google SDK emulator (it's real Android userspace on the host kernel), so it needs a **Linux host kernel with binder/ashmem** modules. On macOS it would run inside a Linux VM; on a Linux x86_64/arm64 gate-host it runs natively no-KVM. This directly enables

the "Linux x86_64 gate host" remediation already recorded in our incident JSONs, AND an arm64-native path.
- **docker-android (HQarroum) / dockerify-android (Shmayro) / sickcodes/dock-droid** — these wrap the **QEMU SDK emulator** in a container (dock-droid uses full QEMU). They still want acceleration for speed and are heavier, but dockerify-android advertises x86 **and arm64** with ADB + **scrcpy web** access. Good for "real Google emulator system image" fidelity when Redroid's non-standard userspace matters.
- Software-mode (TCG, no KVM) SDK emulator is possible but slow — matches the §6.AH "slower-but-reproducible beats fast-but-host-direct" stance.

3b. Screen video recording

- **scrcpy** — server captures screen with **MediaCodec → H.264/AVC**, streams over adb; client decodes with **FFmpeg**. `scrcpy --record file.mp4` records without needing a window; works against `adb connect ip:5555` (i.e. a containerized emulator). This is the cleanest continuous-recording primitive and pairs with §11.4.128 always-on device-recording.
- **`adb screenrecord`** — built-in, no deps, but **max ~3 min per clip** and stops on screen-off; fine for short Challenge captures, needs chunking for long runs.
- **ffmpeg** — post-process/segment/concat scrcpy output; overlay timestamps; extract keyframes as the screenshots we feed to the vision model.

3c. UI driving — Maestro vs Appium vs Espresso

- **Maestro** (open source, YAML flows): black-box, polls for elements up to ~17s, compares screenshots pixel-by-pixel and retries if <0.5% of screen changed, waits ~2s for UI to settle after each action → **built-in flakiness handling**. ~20-30% faster than Appium (hits UIAutomator2 directly, no WebDriver layer), 10x faster authoring via hot-reload. Real result: Todoist Android migrated 63 flows to Maestro early-2025 → **>99% reliability (was ~50%)**, 80min → <20min. **Caveat:** terse errors, **no automatic screenshots/video on failure** (we'd add scrcpy/ffmpeg for that). Best fit for our black-box "drive real UI on the gating matrix" (§6.I/§6.AE).
- **Appium**: most flexible, richest diagnostics (Inspector, logs, screenshots), iOS physical-device support — but WebDriver layer → slower + more flaky (doesn't auto-sync to app internal state / animation completion).
- **Espresso**: our existing `connectedAndroidTest` Challenge harness — white-box, fastest, deterministic, but in-process (can't drive across apps / system dialogs). Keep Espresso as the

constitutional Challenge gate; add Maestro for cross-app + exploratory + the vision-loop driver.

3d. LLM-vision autonomous QA (the "endless fix loop")

- **droidrun / mobilerun** (MIT, `github.com/droidrun/mobilerun`, PyPI `droidrun`) — the strongest ready-made building block. Controls Android (and iOS) via a **Portal app installed over ADB** that fuses **accessibility tree + screenshots**. Flags: `--vision` (send screenshots to LLM), `--vision-only` (screenshot-only, for apps with no a11y tree), `--reasoning` (manager-executor planning). Multi-provider: OpenAI, **Anthropic**, Gemini, Ollama, DeepSeek, OpenRouter, OpenAI-compatible. Install: `uv tool install mobilerun` → `mobilerun setup` → `mobilerun configure` → `mobilerun run "..."`. Raised €2.1M pre-seed (Jul 2025), 3.3k+ stars — actively maintained. **This is our fastest path to "natural-language drive + vision-detect broken UI".**
- **Microsoft OmniParser v2** (`github.com/microsoft/OmniParser`, checkpoints on HF `microsoft/OmniParser-v2.0`, Feb 2025) — pure-vision screen parser: YOLOv8 icon detector + OCR + description model → structured bounding boxes from a raw screenshot, **no a11y tree / HTML needed**. SOTA 39.5% on ScreenSpot-Pro. Use it to turn our scrcpy keyframes into structured "these are the tappable elements + their labels", then let Claude vision reason "the Welcome logo rendered as a white box" / "onboarding completed with 0 providers" — exactly the §6.AB non-crashing-defect class our tests have historically missed. `OmniTool` shows the full parse→vision-model→act loop.
- **ScreenAgent** (IJCAI-24, `niuzaisheng/ScreenAgent`) — reference architecture for the plan→act→**reflect** state machine (VNC screenshots in, mouse/keyboard out) worth copying for our loop's control flow.
- Research signal: "Leveraging LLM Agents for Automated Video Game Testing" (arXiv 2509.22170) — LLM produces a **structured diagnosis report** (goal, attempted actions, why-stuck, screenshot evidence) that can be filed as a ticket. This is the template for auto-generating our §6.O closure logs / LVA workable items from a failed vision run.

Recommended for Lava (Topic 3)

1. **Add a Redroid runner path to the Containers submodule** as the primary no-KVM emulator on a Linux gate-host (closes the practical blocker behind §6.X-debt/§6.AH-debt). Keep the QEMU-SDK-emulator path (dockerify-android style) for Google-system-image fidelity when needed. Honestly record which

runner produced each attestation row (§6.I `runner/runtime/image` fields).
2. **Wire scrcpy `--record` + ffmpeg segmentation** as the always-on recorder (§11.4.128): raw clips git-ignored + codegraph-excluded under the deterministic `<root>/YYYY-MM-DD/<state-hash>/<DEVICE>_<SERIAL>/recording_NNN/` layout; commit only curated evidence. Extract keyframes as the vision-model input.
3. **Pilot droidrun/mobilerun with the Anthropic provider + `--vision`** as an exploratory QA agent driving the debug APK on the Redroid emulator; have it produce a structured defect report per run. Keep Espresso Challenges as the constitutional gate; the vision agent is a **bluff-hunter** that finds the non-crashing UI defects (white logo, wrong nav, gate-bypass) that green tests miss (§6.AB/§6.AK).
4. **Evaluate OmniParser v2** to structure screenshots before the vision call — improves grounding + reduces token cost vs. raw-screenshot-only.
5. **Adopt Maestro** for cross-app + exploratory flows (its flakiness handling is a direct answer to our emulator-flakiness pain), while noting it needs our scrcpy/ffmpeg layer for failure video.

Sources (Topic 3)

- https://github.com/remote-android/redroid-doc
- https://hub.docker.com/r/redroid/redroid
- https://codersera.com/blog/android-emulator-docker-without-kvm/
- https://github.com/HQarroum/docker-android
- https://github.com/Shmayro/dockerify-android
- https://github.com/sickcodes/dock-droid
- https://www.blog.brightcoding.dev/2025/10/04/display-and-control-your-android-device-with-scrcpy-a-comprehensive-guide/
- https://maestro.dev/a/appium-maestro-the-benchmark
- https://devicelab.dev/blog/maestro-vs-appium-2025
- https://github.com/droidrun/mobilerun
- https://github.com/droidrun/mobilerun/blob/main/README.md
- https://pypi.org/project/droidrun/0.2.0/
- https://github.com/microsoft/OmniParser
- https://huggingface.co/microsoft/OmniParser-v2.0
- https://microsoft.github.io/OmniParser/
- https://github.com/niuzaisheng/ScreenAgent
- https://arxiv.org/html/2509.22170v1

Topic 4 — Other open-source pieces for an autonomous mobile-QA system

Structured logcat / crash / ANR analysis

- **logcat.ai** — natural-language Q&A over logcat/bugreport/dmesg/kernel logs; "Deep Research" autonomous agent runs 5-10min multi-step investigations, correlates events across subsystems, cites sources; "Delta" compares log files across app versions to detect regressions and cross-layer root causes. Understands 30+ bugreport/logcat/kernel formats. (Hosted service, but the format-parsing + temporal correlation model is the pattern to mirror in `lava-api-go/internal/qa`.)
- **`lana-20/android-crash-anr-logcat-bugreport`** — open reference for crash-vs-ANR and `adb logcat` vs `adb bugreport` triage; good grounding for our own structured parser.

Visual regression / screenshot-diff (find the "white logo" class deterministically)

- **Lost Pixel** (open source) — flexible config, has explicit **flaky-visual-test handling** guidance (animations are the #1 cause; disable/accelerate before capture).
- **BackstopJS** — screenshot-comparison framework (Playwright/Puppeteer backends).
- **Argos** — surfaces visual diffs directly in PRs, CI-integrated.
- **Percy** (open-source tooling) — AI-driven dynamic-element handling to cut false positives.
- Android-native pairing: Maestro's built-in pixel-diff polling + our Espresso `captureToImage` screenshots. Combine with a **dominant-color / hue assertion** (per §6.AB — RGB-variance alone let the white-logo bug pass) rather than a plain screenshot-equality diff.

Flakiness detection

- Maestro's polling/retry model (Topic 3) is the strongest built-in mobile flakiness mitigation. For test-history-based flake detection, the Semaphore roundup lists the current tooling; the key practice is **quarantine + retry-with-history**, not just pixel diff.

Recommended for Lava (Topic 4)

1. Build a small **structured-logcat parser** in `lava-api-go/internal/qa` (or a Containers-side helper) modeled on logcat.ai's parse→temporal-correlate→diagnose pipeline; feed it the

scrcpy-run logcat to auto-classify crash vs ANR vs warning and attach to the vision agent's defect report.
2. Add **screenshot-diff with a hue/dominant-color assertion** (not bare equality) to the Challenge suite for brand-mark/rendering-correctness — closes the §6.AB discrimination gap that shipped the white-logo defect.
3. Use **Lost Pixel or BackstopJS** as the diff engine if we want PR-surfaced visual diffs; keep them local-only per the Local-Only CI/CD rule.
4. Standardize on **disable/accelerate animations before capture** across all screenshot + Challenge runs to kill the #1 flakiness source.

Sources (Topic 4)

- https://logcat.ai/
- https://blog.logcat.ai/2025/10/05/from-app-crashes-to-kernel-panics-introducing-logcat.ai/
- https://github.com/lana-20/android-crash-anr-logcat-bugreport
- https://www.lost-pixel.com/blog/handling-flaky-visual-regression-tests-with-lost-pixel-platform
- https://www.browserstack.com/guide/visual-regression-testing-open-source
- https://percy.io/blog/open-source-visual-regression-testing-tools
- https://semaphore.io/blog/tools-flaky-tests

Cross-cutting verdict

- **Topic 1:** current fix is correct; add a regression test; adopt `BrotliInterceptor` ONLY if a provider goes br-only AND with a decompression-bomb size cap (CVE-2023-3782 has **no upstream fix**).
- **Topic 2:** Prowlarr Cardigann YAMLs are the field-level source-of-truth; pin UA↔cf_clearance; test decompress→cp1251 ordering; consider a FlareSolverr/Byparr sidecar in `lava-api-go`.
- **Topic 3: Redroid** is the concrete way past the KVM blocker; scrcpy+ffmpeg for recording; **droidrun/mobilerun (MIT) + Claude vision + OmniParser v2** for the autonomous vision loop; Maestro for flakiness-resilient driving.
- **Topic 4:** structured logcat parser + hue-based screenshot-diff + animation-disable close the §6.AB/§6.AK bluff classes deterministically.